

LCS

April 10, 2026

0.1 Longest Common Sequence - Implementazione e Testing

Si consideri il problema della **piu' lunga sottosequenza comune**, una *sottosequenza* di una sequenza s e' una sequenza di *elementi* di s presi nello stesso **ordine**, ossia e' individuata da una sequenza di *indici* di s , non necessariamente **adiacenti**. **Problema:** Date due sequenze trovare la sottosequenza comune di lunghezza massima. (Individuata da una sequenza di coppie di indici).

Iniziamo presentando l'algoritmo **brute force** che, date le due sequenze s_1, s_2 di lunghezza m e n rispettivamente, genera tutte le sottosequenze di s_1 , e per ognuna di esse controlla e questa e' anche sottosequenza di s_2 , tenendo traccia della piu' lunga trovata. Ogni sottosequenza di s_1 corrisponde a un sottoinsieme degli indici della sequenza, quindi esse sono 2^m . Per ognuna occorrono nel caso peggiore n passi per controllare se e' anche sottosequenza di s_2 , utilizzando una scansione simultanea della sottosequenza scelta e di s_2 . La complessita' e' quindi $n \times 2^m$.

La formulazione induttiva della soluzione presentata e' descritta nel modo seguente, dove $S_1[1...m]$ e $S_2[1...n]$ sono le due sequenze e $LCS(i, j)$ indica una sottosequenza comune di lunghezza massima tra i prefissi $S_1[1...i]$ e $S_2[1...j]$.

$$LCS(0, j) = [], \forall j \in \{0, \dots, n\} \quad 0 \leq j \leq n$$

$$LCS(i, 0) = [], \forall i \in \{0, \dots, m\} \quad 0 \leq i \leq m$$

$$\forall i, j \mid i \neq 0, j \neq 0:$$

$$LCS(i, j) = LCS(i-1, j-1) \cdot (i, j) \text{ se } S_1(i) = S_2(j)$$

$$LCS(i, j) = \max(LCS(i-1, j), LCS(i, j-1)) \text{ altrimenti.}$$

Informalmente, come **base** della definizione si ha che la *LCS* di due sequenze di cui una e' vuota risulta vuota. Il **passo induttivo** corrisponde al caso in cui entrambi i prefissi sono non vuoti, si considerano i due casi seguenti: - Se $S_1(i) = S_2(j)$ allora tutte le sottosequenze comuni sono tutte quelle precedenti e tutte quelle ottenute aggiungendo in fondo a una di queste la coppia (i, j) . e dunque $LCS(i, j) = LCS(i-1, j-1) \cdot (i, j)$. - In caso contrario le sottosequenze comuni non potranno contenere la coppia (i, j) . Quindi $LCS(i, j)$ e' la piu' lunga fra le due sequenze $LCS(i-1, j)$ e $LCS(i, j-1)$, questo caso e' il caso piu' complesso che va a generare un numero esponenziale di alberi di ricorsione per valutare la sequenza piu' lunga fra tutte quelle possibili.

La **relazione di ricorrenze**, considerando il *caso peggiore* e' la seguente: $T(n, m) = T(n-1, m) + T(n, m-1) + 1$ e si ha, come per le **Torri di Hanoi** una *complessita' esponenziale* $\Theta(2^n)$.

```
[2]: # Imports
import IPython
import warnings
```

```

PATH = "../data"
F1 = "/dna_sequence_100.txt"
F2 = "/dna_sequence_200.txt"

# Opening and reading files
with open(PATH + F1, 'r') as f1:
    s1 = f1.read()
with open(PATH + F2, 'r') as f2:
    s2 = f2.read()

```

```

[3]: # Alternatively using .py fibonacci sequences
import sys
sys.path.append(PATH)

from code_snippet_1 import fibonacci as fib1
from code_snippet_2 import fibonacci as fib2

N1 = 10
N2 = 12

s1_int = fib1(N1)
s2_int = fib2(N2)

s1 = [str(n) for n in s1_int]
s2 = [str(n) for n in s2_int]

```

```

[4]: # LCS - Brute Force Algorithm

def lcs_bf (s1, s2):
    # From the last character of the strings
    m = len(s1)
    n = len(s2)
    return bf(s1, s2, m, n)

def bf(s1, s2, i, j):
    # Base
    if (i == 0 or j == 0):
        return [] # Empty List
    # Step
    if (s1[i-1] == s2[j-1]):
        last = bf(s1, s2, i-1, j-1)
        # List concatenation
        return last + [(i,j)]
    else:
        lcs_a = bf(s1, s2, i-1, j)
        lcs_b = bf(s1, s2, i, j-1)

```

```

        # The longest List between the two
        return max(lcs_a, lcs_b, key=len)

# Output
res = lcs_bf (s1, s2)
lcs = ""
for i, j in res:
    lcs += s1[i-1]

print(f"LCS Indexes: {res}")
print(f"LCS Length: {len(res)}")
print(f"\nLCS: {lcs}")

```

LCS Indexes: [(1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (6, 6), (7, 7), (8, 8), (9, 9), (10, 10)]

LCS Length: 10

LCS: 0112358132134

Viene fornito un algoritmo migliore con la tecnica della **Programmazione dinamica**, in questo caso e' semplice dimostrare l'applicabilita' di tale approccio in quanto e' presente sia una **Sottostruttura ottima**, ossia che la soluzione del problema contiene le soluzioni dei sottoproblemi necessari alla sua soluzione, che forme di **Sottoproblemi ripetuti**, a causa dei quali avviene l'“*esplosione*” *ricorsiva* e che dunque possiamo memorizzare in strutture dati apposite.

Si costruisce una *matrice LCS* con $m + 1$ righe ed $n + 1$ colonne. In base alla definizione induttiva data sopra, la prima riga e la prima colonna vanno riempite con la sequenza vuota, di lunghezza 0. Si puo' poi procedere riga per riga (o colonna per colonna), l'ultima casella, cioe' quella nell'angolo a destra in basso, conterra' la soluzione. In ogni casella non occorre memorizzare tutta la **LCS** ma basta mettere:

- Se si ha lo stesso carattere sulla riga e sulla colonna, una “*freccia diagonale*”, aumentando di 1 la lunghezza rispetto alla casella puntata dalla freccia;
- Se sulla riga e sulla colonna ci sono due caratteri diversi, una freccia verso la cella di lunghezza maggiore fra la adiacente sopra e la adiacente a sinistra (se hanno uguale lunghezza viene scelto un criterio universale, ad esempio quella sopra).

La lunghezza e' la stessa di quella della cella puntata dalla freccia.

La **LCS** corrispondente a ciascuna casella si ottiene, dall'ultimo elemento al primo, seguendo le frecce a partire dall'ultima casella.

```

[5]: import numpy as np
def lcs_dyn (s1, s2):
    # From the last character of the strings
    m = len(s1)
    n = len(s2)
    L, R = dyn(s1, s2, m, n)

```

```

print("Lenghts Matrix:")
print(L)
print("References Matrix")
print(R)

res = ret_lcs(R, s1)
print(f"\nLCS: {res}")
print(f"LCS Length: {len(res)}")

# Function to create the two matrices, one for lenghts and the other for
↳ references
def dyn(s1, s2, m, n):
    L = np.zeros((m+1, n+1), dtype=int)
    R = np.full((m+1, n+1), ' ', dtype=object)

    for i in range(1, m+1):
        for j in range(1, n+1):
            if (s1[i-1] == s2[j-1]):
                L[i,j] = L[i-1,j-1] + 1
                R[i,j] = ' '
            elif (L[i,j-1] > L[i-1,j]):
                L[i,j] = L[i,j-1]
                R[i,j] = '←'
            else:
                L[i,j] = L[i-1,j]
                R[i,j] = '↑'

    return L, R

# Function to retrieve the LCS
def ret_lcs (R, s1):
    i, j = np.size(R, 0) - 1, np.size(R, 1) - 1
    res = []

    while i > 0 and j > 0:
        if (R[i,j] == ' '):
            res.append(s1[i-1])
            i -= 1
            j -= 1
        elif (R[i,j] == '←'):
            j -= 1
        else:
            i -= 1

    res.reverse() # Its from the last cell
    return res

# Output

```

```
lcs_dyn(s1, s2)
```

Lenghts Matrix:

```
[[ 0  0  0  0  0  0  0  0  0  0  0  0  0]
 [ 0  1  1  1  1  1  1  1  1  1  1  1  1]
 [ 0  1  2  2  2  2  2  2  2  2  2  2  2]
 [ 0  1  2  3  3  3  3  3  3  3  3  3  3]
 [ 0  1  2  3  4  4  4  4  4  4  4  4  4]
 [ 0  1  2  3  4  5  5  5  5  5  5  5  5]
 [ 0  1  2  3  4  5  6  6  6  6  6  6  6]
 [ 0  1  2  3  4  5  6  7  7  7  7  7  7]
 [ 0  1  2  3  4  5  6  7  8  8  8  8  8]
 [ 0  1  2  3  4  5  6  7  8  9  9  9  9]
 [ 0  1  2  3  4  5  6  7  8  9 10 10 10]]
```

References Matrix

[illegible]

LCS: ['0', '1', '1', '2', '3', '5', '8', '13', '21', '34']

LCS Length: 10

Per ricostruire la **LCS** dunque si parte da (m, n) e si va all'indietro seguendo le frecce, in corrispondenza di ogni *freccia diagonale* si ha un elemento della sequenza.

La *complessita' temporale* e' $T(m, n) = \Theta(mn)$ (costruzione di matrici $m \times n$). Assumendo come al solito $m \sim n$, l'algoritmo e' **quadratico**: molto meglio dell'algoritmo **esponenziale**. La *complessita' spaziale* e' anch'essa $S(m, n) = \Theta(mn)$ (memorizzazione di matrici $m \times n$). Se ci interessassimo solo della lunghezza della **LCS** potremmo anche non considerare la matrice dei riferimenti e anche mantenere in memoria solo le tre caselle cointigue $LCS(i-1, j-1), LCS(i-1, j), LCS(i, j-1)$, avendo una *complessita' spaziale lineare* $\Theta(m+n)$ come qui sotto presentato.

```
[6]: import numpy as np
def lcs_dyn2 (s1, s2):
    # From the last character of the strings
    m = len(s1)
    n = len(s2)

    res = dyn2(s1, s2, m, n)
```

```

print(f"LCS Length: {res}")

def dyn2(s1, s2, m, n):
    # I can use two lists to remember the lengths that matter
    prev = np.zeros(n+1)
    cur = np.zeros(n+1)

    for i in range(1, m+1):
        for j in range(1, n+1):
            if (s1[i-1] == s2[j-1]):
                cur[j] = prev[j-1] + 1 # It simulates the diagonal arrow, the
↪upward left element + 1 ( )
            elif (cur[j-1] > prev[j]):
                cur[j] = cur[j-1] # It gets the previous element or the
↪one above (↑,↑)
            else:
                cur[j] = prev[j]
        prev = cur.copy() # It proceeds to the next line
    return prev[n] # Returns the last cell

# Output
lcs_dyn2(s1, s2)

```

LCS Length: 10.0

Com'è possibile notare dai tempi di esecuzione (ad esempio dei file .txt forniti), il primo algoritmo è molto più lento e questo dimostra puntualmente il vantaggio della programmazione dinamica in questo tipo di problemi.

Baldini Filippo - 6393212